

BT-Mux: A Time-Division-Multiplexed Virtual Socket Architecture for High-Density Bluetooth Low Energy Connectivity

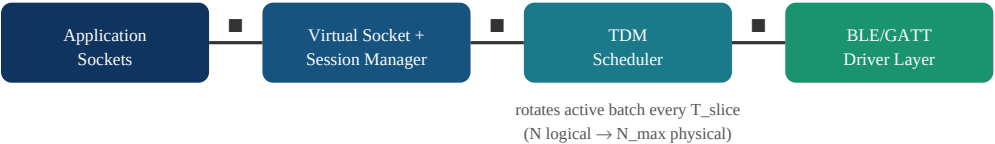
Technical Solution Summary | 3rd-Year B.Tech Project | Indian Institute of Information Technology, Pune

1. Problem Formulation

Commodity Bluetooth Low Energy (BLE) controllers enforce a hard upper bound, $N_{\max}$, on the number of concurrent GATT-level physical links a single radio can sustain, typically in the range of 4-7 for consumer-grade host controllers, and as low as 2 for constrained peripheral controllers such as those found in commercial audio headsets. In practice, however, application demand for independent, persistent logical channels routinely exceeds this ceiling: sensor meshes, multi-peer file-transfer sessions, and multi-source audio consumption scenarios may require $N \geq 10$ simultaneous logical endpoints. This creates a structural mismatch between the connection topology an application expects and the one the underlying radio hardware can physically provide, and it cannot be resolved by firmware modification, since neither host nor peripheral firmware is assumed to be under the developer's control.

2. Proposed Architecture

BT-Mux resolves this mismatch by interposing a user-space middleware layer between the application's socket abstraction and the native BLE driver stack, converting a fixed pool of $N_{\max}$ physical links into an arbitrarily large pool of logical, persistent "virtual sockets." The middleware is decomposed into three cooperating subsystems: a **Session & RAM/FIFO Buffer Manager**, which preserves per-stream state and queues in-flight payloads independently of physical link availability; a **TDM Scheduler**, which rotates the $N_{\max}$ physical slots across batches of logical peripherals on a configurable time window T_{slice} ($100 \text{ ms} \leq T_{\text{slice}} \leq 2000 \text{ ms}$); and a **Diagnostic Telemetry Layer**, which streams scheduler-state transitions over a low-latency IPC channel to a real-time profiling dashboard.



3. Scheduling Model and State Semantics

Each logical socket is governed by a finite-state model: *UNINITIALIZED* $\rightarrow$ *PAUSED/QUEUED* $\rightarrow$ *CONNECTING* $\rightarrow$ *ACTIVE_PHYSICAL_LINK*, cycling back to *PAUSED/QUEUED* on slot-timer expiry, or diverting to *DEGRADED/RETRY* if physical negotiation exceeds a 1000 ms bound. While a socket is queued rather than physically connected, outbound payloads are not rejected; they are retained in a dedicated per-stream FIFO and flushed atomically upon the next *GATT_CONNECTED* event, which is what permits the application layer to treat the channel as continuously available despite the underlying radio being shared. The baseline scheduler operates round-robin over batches of size $N_{\max}$; an extension under evaluation reorders the batch queue using RSSI-guided weighting, granting proportionally larger T_{slice} allocations to links with weaker signal quality or higher outstanding queue depth, so that scarce airtime is spent where it has the greatest marginal effect on delivery ratio.

4. Application-Layer Demonstration and Extensibility

The core architecture is application-agnostic: it multiplexes byte streams, not any particular payload semantics. The primary evaluation demonstration is a **multi-peer file-transfer testbed**, in which $N \geq 10$ simulated peripheral nodes concurrently exchange chunked file payloads through the virtual socket layer, allowing objective, reproducible measurement of throughput and reliability under TDM rotation. The identical scheduling and buffering core generalizes directly to constrained-peripheral scenarios — for instance, a relay interposed between N source devices and a two-connection-limited audio peripheral — by substituting the GATT transport for a Classic Bluetooth A2DP/AVRCP transport and replacing round-robin slot rotation with activity-triggered source arbitration. This variant is scoped as a forward extension beyond the present benchmarking phase.

It is worth distinguishing this extension from an existing, superficially similar product class: commercial "silent disco" / silent-DJ systems already solve the *inverse* topology — one audio source broadcast to many independent headphone receivers. What the BT-Mux relay targets is the complementary and considerably less common case, **one constrained headphone accepting many candidate sources**, where the relay continuously holds the peripheral's single physical audio link while N logical source connections remain queued behind it. Since only one source can be audible at a time, the relay additionally requires a **source-selection policy** to arbitrate which queued source is promoted to the active link; the baseline policy promotes whichever source most recently began streaming audio, with priority-rank or manual-override policies identified as straightforward extensions of the existing *session_nodes.priority_rank* field already present in the data model.

5. Evaluation Criteria

Metric	Target	Rationale
Packet Delivery Ratio (PDR)	$\geq 95\% \text{ @ } N=10$	Confirms buffering masks slot rotation losslessly

Context-switch handoff latency	≤ 300 ms	Bounds perceptible discontinuity per slot swap
Peripheral acquisition timeout	1000 ms $\rightarrow$ DEGRADED	Prevents one unstable node stalling the schedule
Steady-state memory footprint	< 150 MB	Keeps middleware viable on commodity host hardware

In summary, BT-Mux reframes a fixed hardware connection ceiling as a scheduling problem rather than a hardware problem: by decoupling logical session lifetime from physical link lifetime through buffered, time-division-scheduled virtual sockets, it allows a constrained radio to service an arbitrarily larger set of concurrent logical peers, with the file-transfer testbed serving as the quantitatively benchmarked proof of concept for this mechanism.